Intelligent Payment Fraud Detection and Risk-Based Decision System

Track: Data Science and Artificial Intelligence

Contents

- **Part 1 — Project Planning & Management**
- **Part 2 — Literature Review**
- **Part 3 — Requirements Gathering**

Part 1 — Project Planning & Management

1.1 Project Overview

Project Name: Intelligent Payment Fraud Detection and Risk-Based Decision System **Project Track:** Data Science and Artificial Intelligence

The project aims to develop a Machine Learning-based system that detects potentially fraudulent financial transactions, assigns risk scores, and explains the main factors influencing its predictions. The system will also support risk-based decisions by prioritizing suspicious transactions for further investigation.

The project will use the IEEE-CIS Fraud Detection dataset from Kaggle and deliver a trained Machine Learning model alongside an interactive dashboard.

1.2 Problem Statement

Financial fraud detection is challenging because fraudulent transactions represent a small proportion of total transactions. This class imbalance can lead to misleading accuracy scores and poor fraud detection performance. Additionally, false alarms and missed fraudulent transactions can result in financial losses and unnecessary investigation costs.

The project addresses these challenges by combining fraud detection, explainable predictions, and risk-based decision support.

1.3 Project Objectives

- Analyze and preprocess financial transaction data.
- Develop and compare Machine Learning models for fraud detection.
- Evaluate performance using appropriate metrics for imbalanced data.
- Generate risk scores and explain model predictions.
- Build an interactive dashboard to visualize suspicious transactions and model results.
- Analyze the trade-off between detecting fraud, false alarms, and missed cases.

1.4 Project Scope

In Scope:

- Data preprocessing and Exploratory Data Analysis (EDA).
- Machine Learning model development and comparison.
- Model evaluation using Precision, Recall, F1-score, and PR-AUC.
- Risk scoring, model explainability, and decision-cost analysis.
- Interactive dashboard development and system testing.

Out of Scope:

- Direct integration with real banking systems.
- Automatically blocking real transactions.
- Guaranteeing fraud detection or claiming validated performance on Egyptian banking data.

The project will be an academic prototype and decision-support tool.

1.5 Project Methodology

The project will follow these main stages:

1. Dataset exploration and literature review.
2. Data cleaning, preprocessing, and feature engineering.

3. Exploratory data analysis.
4. Baseline model development and comparison with alternative models.
5. Model evaluation and improvement.
6. Risk scoring and prediction explainability.
7. Dashboard development and integration.
8. Testing, documentation, and final presentation.

1.6 Project Timeline

The following is a proposed eight-week plan, subject to the official project schedule.

Week	Activities
1	Planning, dataset exploration, and literature review
2	Data preprocessing and EDA
3	Baseline model development
4	Model comparison and class imbalance handling
5	Model evaluation and improvement
6	Explainability and risk-based decision support
7	Dashboard development and integration
8	Testing, documentation, and presentation

1.7 Team Roles and Responsibilities

The four team members will share responsibilities as follows:

- **Member 1 — Data Analysis:** Data exploration, cleaning, preprocessing, and EDA.
- **Member 2 — Machine Learning:** Model development, training, and comparison.
- **Member 3 — Evaluation and Explainability:** Performance metrics, model interpretation, and decision-cost analysis.
- **Member 4 — Dashboard and Integration:** Dashboard development, visualization, and system integration.

All members will contribute to documentation, testing, and the final presentation.

1.8 Risk Management

Risk	Mitigation
Class imbalance	Use appropriate metrics and test suitable balancing techniques.
Overfitting or data leakage	Apply proper validation and data-splitting procedures.
High false positive rate	Analyze Precision, false alarms, and decision thresholds.
Missed fraudulent transactions	Monitor Recall and investigate false negatives.
Limited computational resources	Start with baseline models before testing more complex approaches.
Limited applicability to Egypt	Document dataset limitations and avoid unsupported claims about local performance.

1.9 Success Criteria and KPIs

Project success will be measured using:

- **Model Performance:** Precision, Recall, F1-score, PR-AUC, and False Positive Rate.
- **Decision Support:** Comparison of detected fraud, missed cases, and estimated decision costs.
- **System Functionality:** Successful predictions, risk scoring, explainability, and dashboard visualization.

- **Project Delivery:** Completion of the prototype, testing, documentation, and presentation.

Numerical performance targets will be determined after initial experiments, and results will only be reported after testing.

1.10 Expected Deliverables

The expected deliverables include:

1. A data preprocessing pipeline.
2. A trained and evaluated fraud detection model.
3. A model evaluation report.
4. A risk-scoring and decision-support component.
5. An interactive dashboard.
6. A shared GitHub repository containing the source code and documentation.

1.11 Conclusion

The project combines Machine Learning-based fraud detection with explainable risk assessment and decision support. The final prototype will demonstrate how suspicious transactions can be identified, evaluated, and prioritized for review while highlighting the limitations of the available data and model predictions.

Part 2 — Literature Review

2.1 Review of Existing Solutions

2.1.1 Kaggle Winning Solutions (IEEE-CIS Competition)

Description: The first-place team (including Chris Deotte and Konstantin Yakovlev) combined XGBoost, CatBoost, and LightGBM models. Their key innovation was feature engineering: group-aggregation statistics (mean, standard deviation, counts) and a derived client identifier (`UID`, built from `card1`, `addr1`, and `D1`). Time-ordered GroupKFold validation was used, and predictions were post-processed by averaging them within each client group. The reported final scores were public LB 0.9677 and private LB 0.9459 (ROC-AUC) [1].

What they did well (what we leverage):

- Aggregation-based behavioural features that measure whether a transaction is rare or common for a card or client [1].
- Time-respecting validation, which we adopt to mitigate the leakage risk identified in Part 1 [1].
- Systematic feature selection. The V-column groups were reduced by NaN structure and correlation ($r > 0.75$) [1].

What they lacked (what we avoid):

- The objective was leaderboard ROC-AUC. ROC-AUC can look optimistic under severe imbalance, and no decision threshold or cost model was produced [1].
- Multi-model ensembles with hundreds of engineered features give no per-transaction explanation. Only global feature-importance charts were reported [1].
- Client-level post-processing depends on a dataset-specific identifier reconstruction.

2.1.2 Standard ML and EDA Notebooks

Description: Public notebooks, such as Artgor's "EDA and Models" [2], typically combine data exploration, basic feature engineering, and gradient-boosted models evaluated on the competition metric.

What they did well (what we leverage):

- Visual diagnosis of missingness, class imbalance, and distributions, which supports our Weeks 1–2 EDA phase.
- Reproducible, accessible baselines that show which model families are competitive.

What they lacked (what we avoid):

- Evaluation typically stops at ROC-AUC or accuracy, with no precision/recall trade-off, no cost analysis, and no investigator-facing output.
- No explanation layer and no deployment-oriented component.

2.1.3 Traditional Rule-Based Banking Systems

Description: Rule engines use fixed, human-written conditions (amount, velocity, geography, device) to approve, review, or block transactions [3], [5].

What they do well (what we leverage):

- High transparency. Every alert maps to an explicit rule, which also suits compliance and reporting needs [3], [5].
- Immediate control over known fraud patterns.

What they lack (what we avoid):

- Static thresholds do not learn and adapt slowly to new fraud tactics [3].
- Rule sets that grow and interact can conflict, producing both false negatives and false positives with operational cost [3].

- Excess false declines increase manual review and harm customer experience [5].

2.2 Comparative Analysis

Entries for "Our Proposed System" are **design targets, not validated results**. Consistent with Part 1, results will be reported only after testing.

Criterion	Kaggle Winning Solutions	Standard ML/EDA Notebooks	Rule-Based Banking Systems	Our Proposed System
Adaptability to new patterns	High (retrainable, learned features)	Medium (retrainable, limited engineering)	Low (manual rule updates)	High (retrainable ML pipeline)
False-positive handling	Not addressed (no threshold or cost analysis)	Not addressed	Weak (static thresholds, high review load)	Explicit (threshold tuning, decision-cost analysis)
Explainability	Low (ensemble, global importance only)	Low–Medium (feature importance)	High (explicit rules)	Medium–High (SHAP per-transaction; limited by masked features)
Business decision support	None (leaderboard objective)	None	Binary approve/review/block	Prioritised risk score for investigators
Imbalance-aware evaluation	ROC-AUC only	ROC-AUC / accuracy	Not applicable	Precision, Recall, F1, PR-AUC
Deployment practicality	Low (complex, competition-tuned)	Low	High	Prototype only (academic scope)

2.3 Research Gap and Our Contribution

Gap. The three approaches each solve one dimension. Competition solutions maximise predictive power but ignore explanation and decision cost. Notebooks aid understanding but do not produce decisions. Rule engines are transparent but inflexible and costly in false positives. None of the reviewed approaches combines imbalance-aware learning, per-transaction explanation, and an investigator-oriented risk ranking.

Contribution. Our system targets this combination:

1. **Robust ML for imbalance:** gradient-boosted baselines, tested balancing techniques, and time-based validation, evaluated with PR-AUC, Precision, and Recall.
2. **Transparency through XAI:** SHAP-based explanations [6] for individual predictions, so an investigator sees why a transaction was flagged.
3. **Risk scoring for human review:** calibrated probabilities converted into a ranked queue, with thresholds chosen through explicit false-positive and false-negative cost analysis.

Limitations (stated for academic honesty, consistent with Part 1).

- Anonymised features may make SHAP explanations technically valid but hard to translate into business language.
- The dataset is not Egyptian banking data, so no claims about local performance are made.
- Cost parameters will be assumptions, not bank-validated figures.

References

[1] C. McDonald and C. Deotte, "Leveraging Machine Learning to Detect Fraud: Tips to Developing a Winning Kaggle Solution," NVIDIA Technical Blog, Jan. 2021. https://developer.nvidia.com/blog/leveraging-machine-learning-to-detect-fraud-tips-to-developing-a-winning-kaggle-solution/

[2] Artgor, "EDA and models," Kaggle Notebook, IEEE-CIS Fraud Detection. https://www.kaggle.com/code/artgor/eda

-and-models

[3] Springer chapter on limitations of rule-based fraud detection systems and AI-based alternatives. https://link.springer.com/chapter/10.1007/978-3-030-94590-9_15

[4] IEEE Computational Intelligence Society and Vesta Corporation, “IEEE-CIS Fraud Detection,” Kaggle competition. https://www.kaggle.com/c/ieee-fraud-detection

[5] Signifyd, “Rules-based vs. machine learning in fraud protection”. https://www.signifyd.com/?p=59779

[6] S. M. Lundberg and S.-I. Lee, “A Unified Approach to Interpreting Model Predictions,” Advances in Neural Information Processing Systems (NeurIPS), 2017.

Part 3 — Requirements Gathering

3.1 Stakeholder Identification

The project is designed to support users involved in financial transaction monitoring and fraud investigation. The main stakeholders include:

- **Financial Institutions:** Banks and payment service providers interested in reducing fraud-related losses.
- **Fraud Analysts:** Professionals responsible for reviewing suspicious transactions and investigating potential fraud.
- **Risk Management Teams:** Teams that assess financial risks and establish transaction review policies.
- **System Administrators:** Users responsible for managing system access, data handling, and system operation.

3.2 User Needs

The identified stakeholders require a system that can identify potentially fraudulent transactions, prioritize suspicious cases, and provide understandable explanations for its predictions. The system should also help users understand the trade-off between detecting fraudulent transactions and incorrectly flagging legitimate ones. Results should be presented through a clear dashboard that supports analysis and informed decision-making.

3.3 Functional Requirements

The system shall provide the following functionalities:

1. **Data Input:** Load transaction data from supported files, such as CSV.
2. **Data Preprocessing:** Handle missing values, prepare features, and validate input data before analysis.
3. **Fraud Detection:** Use a trained Machine Learning model to estimate the likelihood that a transaction is fraudulent.
4. **Risk Scoring:** Assign a risk score and categorize transactions into risk levels according to configurable thresholds.
5. **Prediction Explainability:** Present the main features influencing selected predictions using an appropriate explainability method.
6. **Dashboard Visualization:** Display transaction statistics, risk distributions, suspicious transactions, and model performance metrics.
7. **Model Evaluation:** Report relevant metrics, including Precision, Recall, F1-score, and PR-AUC.
8. **Decision Support:** Allow users to explore how different risk thresholds affect fraud detection, false alarms, and missed fraudulent transactions.

3.4 Non-Functional Requirements

- **Performance:** Process supported datasets and generate predictions within a reasonable time using the available hardware.
- **Reliability:** Evaluate the model on unseen data and maintain consistent processing procedures.
- **Security and Privacy:** Restrict access to sensitive data, avoid exposing unnecessary transaction information, and handle files securely.
- **Usability:** Provide a clear, intuitive dashboard with understandable labels, visualizations, and results.
- **Maintainability:** Organize the source code into modular components to simplify testing and future updates.
- **Scalability:** Design the data-processing and prediction pipeline so that it can be extended to larger datasets when resources permit.
- **Interpretability:** Present prediction explanations in a form that users can understand without implying that the model proves fraudulent intent.

3.5 Data Requirements

The system will primarily use the IEEE-CIS Fraud Detection dataset available on Kaggle. The required data includes transaction-related features and the corresponding fraud labels available for model training and evaluation.

The dataset must be inspected to identify missing values, categorical variables, class imbalance, and potential data leakage. Data preprocessing and feature selection must be performed carefully. Training and evaluation procedures must prevent information from the validation or test data from leaking into model training.

Because the dataset is not specific to Egypt, its results will not be treated as evidence of equivalent performance on Egyptian banking transactions.

3.6 Technical Requirements

The project will use Python as the primary programming language, supported by suitable Data Science and Machine Learning libraries, such as Pandas, NumPy, and Scikit-learn. Additional tools may include visualization libraries, an appropriate explainability library, and a framework such as Streamlit for the interactive dashboard.

Git and GitHub will be used for version control and collaborative development. The final technology stack will depend on the team’s implementation choices and available computational resources.

3.7 Constraints and Assumptions

- The project depends on the availability and suitability of the public IEEE-CIS dataset.
- Development is limited by the team’s schedule, skills, and computational resources.
- Model predictions depend on the quality and representativeness of the available data.
- The system will operate as an academic prototype and will not directly authorize, reject, or block real financial transactions.
- Risk scores and explanations will support investigation rather than provide definitive proof of fraud.

3.8 Acceptance Criteria

The project will be considered functionally complete when:

1. The system successfully loads and preprocesses supported transaction data.
2. A trained model generates fraud predictions and risk scores.
3. Model performance is evaluated using appropriate metrics on unseen data.
4. The dashboard displays predictions, risk levels, and relevant evaluation results.
5. An explainability method provides feature-based insights for selected predictions.
6. The system demonstrates how decision thresholds affect false positives and missed fraud cases.
7. The source code, testing results, and technical limitations are documented in the team’s GitHub repository.

3.9 Conclusion

These requirements define the expected functionality, quality attributes, data needs, and technical constraints of the Intelligent Payment Fraud Detection and Risk-Based Decision System. They will guide system development and provide a clear basis for testing whether the final prototype meets its intended objectives.